

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN**

ĐH * ĐHTT



**ĐỒ ÁN MÔN HỌC
KHAI THÁC DỮ LIỆU
ĐỀ TÀI
TRIỂN KHAI CÁC THUẬT TOÁN KHAI PHÁ
DỮ LIỆU**

Lớp: IS252.P11

Giảng viên hướng dẫn: ThS. Mai Xuân Hùng

Sinh viên thực hiện:

1. Nguyễn Cao Thắng

22521329

TP. HỒ CHÍ MINH – 6/2025

LỜI CẢM ƠN

Tôi xin chân thành cảm ơn Trường Đại học Công nghệ Thông tin và Khoa Hệ thống Thông tin đã tạo điều kiện thuận lợi để tôi hoàn thành đồ án môn học *Khai thác dữ liệu*.

Trong quá trình học tập và thực hiện đồ án, tôi đã tích lũy được nhiều kiến thức chuyên môn cũng như kinh nghiệm thực tiễn quý báu. Đặc biệt, tôi xin gửi lời biết ơn sâu sắc đến thầy **Mai Xuân Hùng** – giảng viên phụ trách môn *Khai thác dữ liệu (IS252.P21)* – đã tận tình hướng dẫn, dành thời gian đóng góp ý kiến giúp tôi hoàn thiện tốt bài báo cáo.

Thông qua quá trình thực hiện đề tài, tôi đã củng cố kiến thức về khai thác dữ liệu, đồng thời học thêm cách sử dụng các công cụ hỗ trợ phân tích, góp phần nâng cao năng lực chuyên môn cho bản thân. Bên cạnh đó, việc tự thực hiện đề tài cũng giúp tôi rèn luyện tính tự học, khả năng nghiên cứu và giải quyết vấn đề – những kỹ năng rất quan trọng cho công việc sau này.

Do hạn chế về thời gian và kinh nghiệm, bài làm chắc chắn không tránh khỏi những thiếu sót. Tôi rất mong nhận được những góp ý quý báu từ thầy để đề tài được hoàn thiện hơn.

Trân trọng cảm ơn!

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting or typing. There are no margins, text, or other markings on the page.

MỤC LỤC

LỜI CẢM ƠN	1
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN.....	2
MỤC LỤC	3
MỤC LỤC BẢNG.....	6
MỤC LỤC ẢNH	7
TÓM TẮT NỘI DUNG	8
PROJECT IMPLEMENTATION OVERVIEW	9
Chương 1: GIỚI THIỆU.....	1
1.1. Đặt vấn đề.....	1
1.2. Mục tiêu.....	1
1.3. Phạm vi.....	1
Chương 2: CƠ SỞ LÝ THUYẾT.....	3
2.1. Khai phá dữ liệu	3
2.2. Một số hướng chính của khai phá dữ liệu	3
2.2.1. Tập phổ biến và luật kết hợp	4
2.2.2. Tập thô (Rough Sets).....	5
2.2.3. Phân lớp (Classification)	6
2.2.4. Gom cụm (Clustering).....	8
2.3. Mô hình MVC (Model-View-Controller)	9
2.3.1. Các thành phần chính của MVC.....	10
2.3.2. Luồng hoạt động.....	10
2.3.3. Ưu và nhược điểm của kiến trúc	10
Chương 3: TRIỂN KHAI CÀI ĐẶT.....	11
3.1. Cài đặt theo mô hình MVC	11

3.1.1. Model.....	11
3.1.2. View.....	12
3.1.3. Controller.....	13
3.2. Chức năng: Tính tương quan.....	14
3.2.1. Mô tả.....	14
3.2.2. Giao diện và kiểm tra	15
3.3. Chức năng: Chuẩn hóa Min-Max.....	16
3.3.1. Mô tả.....	16
3.3.2. Giao diện và kiểm tra	17
3.4. Chức năng: Tập phổ biến và luật kết hợp.....	17
3.4.1. Mô tả.....	17
3.4.2. Giao diện và kiểm tra	18
3.5. Chức năng: Tập thô	20
3.5.1. Mô tả.....	20
3.5.2. Giao diện và kiểm tra	21
3.6. Chức năng: Cây quyết định.....	22
3.6.1. Mô tả.....	22
3.6.2. Giao diện và kiểm tra	23
3.7. Chức năng: Phân lớp Naive Bayes.....	24
3.7.1. Mô tả.....	24
3.7.2. Giao diện và kiểm tra	25
3.8. Chức năng: Phân cụm K-means	26
3.8.1. Mô tả.....	26
3.8.2. Giao diện và kiểm tra	27
3.9. Chức năng: Phân cụm Kohonen Self-Organizing Map (SOM)	28

3.9.1. Mô tả.....	28
3.9.2. Giao diện và kiểm tra	29
KẾT LUẬN	31

MỤC LỤC BẢNG

Table 1. Dữ liệu trong file du_lieu_xemay_tuong_quan.csv	15
Table 2. Dữ liệu trong file du_lieu_hoa_don_tap_pho_bien.csv	18
Table 3. Dữ liệu trong file du_lieu_hinh_dang_tap_tho.csv	21
Table 4. Dữ liệu trong file du_lieu_thoi_tiet_cay_quyet_dinh.csv	23
Table 5. Dữ liệu trong file du_lieu_thoi_tiet_bayes.csv	25
Table 6. Dữ liệu trong file du_lieu_diem_k_mean.csv	28
Table 7. Dữ liệu trong file du_lieu_tranh_kohonen.csv	30

MỤC LỤC ẢNH

Hình 1. Cấu trúc thư mục	11
Hình 2. Kết quả thực hiện kiểm thử chức năng tính tương quan	16
Hình 3. Kết quả thực hiện kiểm thử chức năng chuẩn hóa Min – Max	17
Hình 4. Kết quả thực hiện kiểm thử chức năng tập phổ biến và luật kết hợp	19
Hình 5. Kết quả thực hiện kiểm thử chức năng tập thô.....	21
Hình 6. Kết quả thực hiện kiểm thử chức năng cây quyết định	24
Hình 7. Kết quả thực hiện kiểm thử chức năng phân lớp Naive Bayes	26
Hình 8. Kết quả thực hiện kiểm thử chức năng phân cụm K-means.....	28
Hình 9. Kết quả thực hiện kiểm thử chức năng phân cụm Kohonen SOM.....	30

TÓM TẮT NỘI DUNG

Đồ án này được thực hiện với mục tiêu áp dụng các kỹ thuật cốt lõi trong lĩnh vực *khai thác dữ liệu* nhằm xây dựng một hệ thống có khả năng xử lý, phân tích và trực quan hóa thông tin một cách hiệu quả. Python được sử dụng làm ngôn ngữ lập trình chính nhờ sự linh hoạt và hệ sinh thái thư viện phong phú hỗ trợ cho xử lý dữ liệu. Các thư viện như pandas, numpy, itertools và matplotlib đóng vai trò quan trọng trong việc đọc dữ liệu, tiền xử lý, xây dựng mô hình, huấn luyện và trực quan hóa kết quả. Việc sử dụng các công cụ này không chỉ giúp tăng hiệu suất làm việc mà còn thể hiện khả năng tích hợp giữa lý thuyết và thực hành.

Trong phần cài đặt, đồ án tập trung triển khai một số thuật toán tiêu biểu bao gồm:

- **Thuật toán Apriori:** Dùng để khai thác luật kết hợp, hỗ trợ việc tìm kiếm mối liên hệ giữa các mục dữ liệu trong tập giao dịch.
- **Lý thuyết tập thô:** Áp dụng các khái niệm như xấp xỉ tập hợp, phụ thuộc thuộc tính, reduct và luật có độ chính xác tuyệt đối để khai thác tri thức từ dữ liệu không đầy đủ.
- **Gom cụm K-Means:** Giúp phân nhóm dữ liệu dựa trên khoảng cách tương đồng, hỗ trợ khám phá cấu trúc tiềm ẩn bên trong tập dữ liệu.
- **Phân lớp bằng cây quyết định:** Cho phép tạo mô hình dễ hiểu, dễ diễn giải, phù hợp với các bài toán phân loại có tính trực quan.
- **Phân lớp bằng mạng Bayes:** Áp dụng lý thuyết xác suất để dự đoán thuộc tính mục tiêu dựa trên mối quan hệ giữa các đặc trưng.
- **Gom cụm bằng mạng Kohonen (Self-Organizing Map):** Một kỹ thuật học không giám sát với khả năng biểu diễn dữ liệu đa chiều trên mặt phẳng hai chiều dễ quan sát.

Sau khi cài đặt và kiểm thử các thuật toán, tôi sử dụng thư viện tkinter để xây dựng giao diện người dùng thân thiện. Giao diện này cho phép người dùng thao tác trực tiếp như:

- Tải dữ liệu đầu vào dưới dạng file Excel hoặc CSV.
- Tùy chọn tham số thuật toán (ví dụ: ngưỡng hỗ trợ, độ tin cậy, số lượng cụm, thuộc tính mục tiêu...).
- Thực thi thuật toán và xem kết quả trực quan ngay trên cửa sổ ứng dụng.

Kết quả đầu ra của từng thuật toán được trình bày dưới nhiều định dạng khác nhau như biểu đồ và các chỉ số đánh giá. Việc này giúp người dùng dễ dàng phân tích, đánh giá và so sánh hiệu quả của các phương pháp được triển khai.

PROJECT IMPLEMENTATION OVERVIEW

This project was carried out with the goal of applying core techniques in the field of *data mining* to develop a system capable of efficiently processing, analyzing, and visualizing information. Python was chosen as the main programming language due to its flexibility and the rich ecosystem of libraries that support data analysis tasks. Key libraries such as pandas, numpy, itertools and matplotlib played essential roles in data loading, preprocessing, model building, training, and visualization. The use of these tools not only improved development efficiency but also highlighted the ability to bridge theory with practical implementation.

In terms of technical implementation, the project focuses on several representative algorithms, including:

- **Apriori Algorithm:** Used for mining association rules, enabling the discovery of relationships between items in transaction datasets.
- **Rough Set Theory:** Applies concepts such as set approximation, attribute dependency, reducts, and 100% accuracy rules to extract knowledge from incomplete or imprecise data.
- **K-Means Clustering:** Groups data based on similarity metrics, helping to uncover hidden structures within datasets.
- **Decision Tree Classification:** Builds interpretable models suited for classification problems with high visual clarity.
- **Bayesian Network Classification:** Utilizes probabilistic reasoning to predict target attributes based on the relationships among features.
- **Kohonen Self-Organizing Map (SOM):** An unsupervised learning technique that maps high-dimensional data onto a two-dimensional plane for easier interpretation.

After implementing and testing the algorithms, a user-friendly graphical interface was developed using the **Tkinter** library. This interface allows users to interact directly with the application through features such as:

- Uploading input datasets in Excel or CSV format
- Selecting algorithm parameters (e.g., support threshold, confidence level, number of clusters, target attributes, etc.)
- Running the algorithms and viewing results directly in the application window

The output for each algorithm is presented in multiple formats, including visual charts and evaluation metrics, enabling users to easily analyze, assess, and compare the effectiveness of the implemented methods.

Chương 1: GIỚI THIỆU

1.1. Đặt vấn đề

Cuộc cách mạng dữ liệu đang diễn ra mạnh mẽ trên toàn cầu, đặt ra những thách thức lớn trong việc khai thác giá trị từ khối lượng thông tin khổng lồ được tạo ra hàng ngày. Khai phá dữ liệu (Data Mining) xuất hiện như một giải pháp mang tính đột phá, giúp các tổ chức không chỉ hiểu rõ hơn về dữ liệu hiện có mà còn phát hiện những mẫu hình ẩn giấu có thể định hình tương lai.

Sự phát triển vượt bậc của Big Data đã thúc đẩy nhu cầu về các công cụ phân tích tinh vi. Từ việc dự báo hành vi khách hàng trong thương mại điện tử, chẩn đoán y tế thông minh, đến tối ưu hóa hệ thống giáo dục - khai phá dữ liệu đã trở thành xương sống của quá trình ra quyết định thông minh.

Python đã khẳng định vị thế dẫn đầu trong lĩnh vực này với một hệ sinh thái công cụ đa dạng và mạnh mẽ. Sự kết hợp hiệu quả giữa pandas cho xử lý dữ liệu, numpy cho tính toán hiệu suất cao, itertools cho việc tạo ra các tổ hợp và lặp dữ liệu phức tạp, cùng với matplotlib cho trực quan hóa đã tạo nên một nền tảng phát triển ứng dụng hoàn chỉnh. Đặc biệt, tkinter - thư viện giao diện đồ họa tích hợp sẵn của Python - mang đến khả năng xây dựng ứng dụng desktop trực quan và thân thiện với người dùng.

Nhận thức được tầm quan trọng này, đề tài "Triển Khai Các Thuật Toán Khai Phá Dữ Liệu" được thực hiện với mục đích xây dựng một cầu nối giữa lý thuyết và thực hành, từ các khái niệm cơ bản đến các kỹ thuật chuyên sâu như phân cụm, phân lớp và khám phá luật kết hợp.

1.2. Mục tiêu

Đề tài hướng đến việc thực hiện ba mục tiêu chính:

- **Xây dựng nền tảng lý thuyết vững chắc:** Tổng hợp và trình bày một cách có hệ thống các kỹ thuật khai phá dữ liệu, từ cơ bản đến nâng cao, kèm theo việc triển khai thực tế các thuật toán quan trọng.
- **Phát triển ứng dụng desktop tương tác:** Xây dựng giao diện người dùng trực quan bằng tkinter, cho phép người dùng dễ dàng tải lên dữ liệu, điều chỉnh các tham số thuật toán và quan sát kết quả một cách trực tiếp trên máy tính cá nhân.
- **Thiết lập hệ thống đánh giá toàn diện:** Xây dựng framework đánh giá hiệu suất đa chiều, giúp người dùng có cái nhìn khách quan về chất lượng của các mô hình được áp dụng.

1.3. Phạm vi

Đề tài được giới hạn trong các khía cạnh sau:

- **Công nghệ sử dụng:** Python được chọn làm ngôn ngữ chính với những ưu điểm vượt trội về tính đơn giản, khả năng mở rộng và cộng đồng hỗ trợ mạnh mẽ. Đề tài kết hợp với bộ thư viện cốt lõi được lựa chọn cẩn thận gồm

pandas (xử lý và phân tích dữ liệu cấu trúc), numpy (tính toán khoa học và xử lý mảng đa chiều), itertools (tạo iterator hiệu quả cho các phép toán tổ hợp), matplotlib (trực quan hóa dữ liệu với đa dạng loại biểu đồ) và tkinter (phát triển giao diện đồ họa desktop) để đảm bảo tính nhất quán, hiệu quả và khả năng triển khai độc lập.

- **Hình thức trình bày kết quả:** Thông tin và kết quả phân tích được thể hiện thông qua nhiều định dạng phong phú bao gồm bảng dữ liệu có cấu trúc với khả năng sắp xếp và lọc, biểu đồ trực quan đa dạng (scatter plot, histogram, heatmap, dendrogram), các chỉ số phân tích định lượng được tính toán tự động, và báo cáo tổng hợp có thể xuất ra file. Tất cả được tích hợp trong một giao diện thống nhất, tạo nên hệ thống báo cáo đầy đủ, dễ hiểu và có tính tương tác cao.
- **Giá trị mang lại:** Đề tài không chỉ đóng vai trò như một tài liệu tham khảo lý thuyết toàn diện mà còn là một công cụ thực hành hiệu quả, giúp người dùng có thể tiếp cận, học hỏi và áp dụng các kỹ thuật khai phá dữ liệu một cách trực quan và chính xác. Thông qua giao diện thân thiện và các tính năng tương tác, đề tài góp phần đơn giản hóa việc tiếp cận công nghệ khai phá dữ liệu, giúp người dùng đưa ra các quyết định dựa trên dữ liệu một cách hiệu quả và tin cậy nhất.

Chương 2: CƠ SỞ LÝ THUYẾT.

2.1. Khai phá dữ liệu

Khai phá dữ liệu (Data mining) là quá trình sử dụng các phương pháp máy học, thống kê và xử lý dữ liệu nhằm tự động tìm ra các quy luật, mẫu ẩn và mối quan hệ có ý nghĩa trong các bộ dữ liệu lớn và phức tạp. Đây là một bước quan trọng trong quy trình Khám phá tri thức trên cơ sở dữ liệu (Knowledge Discovery in Databases - KDD). Quá trình KDD bao gồm bảy bước tuần tự như sau:

1. Làm sạch dữ liệu (Data cleaning & preprocessing): Loại bỏ các dữ liệu nhiễu, trùng lặp hoặc không cần thiết.
2. Tích hợp dữ liệu (Data integration): Kết hợp dữ liệu từ các nguồn khác nhau (kho dữ liệu, data marts) sau khi đã làm sạch và tiền xử lý.
3. Trích chọn dữ liệu (Data selection): Lọc và trích xuất một tập con dữ liệu phù hợp từ kho dữ liệu để phân tích tiếp, đồng thời xử lý các giá trị thiếu hoặc nhiễu.
4. Biến đổi dữ liệu (Data transformation): Chuyển đổi dữ liệu sang dạng phù hợp cho bước khai phá tiếp theo (ví dụ: chuẩn hóa, chiếu xuống thuộc tính, ghép nhóm).
5. Khai phá dữ liệu (Data mining): Áp dụng các thuật toán thông minh để khám phá các mô hình, quy luật có giá trị từ dữ liệu.
6. Đánh giá mẫu (Knowledge evaluation): Đánh giá các kết quả khai phá bằng các tiêu chí phù hợp để chọn ra các quy tắc, mô hình hữu ích.
7. Trình bày tri thức (Knowledge presentation): Sử dụng các kỹ thuật trực quan hóa (biểu đồ, báo cáo, ...) để thể hiện kết quả sao cho người dùng cuối có thể hiểu và áp dụng.

Khai phá dữ liệu giúp biến dữ liệu thô thành thông tin và tri thức có giá trị, hỗ trợ việc ra quyết định trong nhiều lĩnh vực như thương mại điện tử, phân tích khách hàng, y tế và khoa học.

2.2. Một số hướng chính của khai phá dữ liệu

Trong khai phá dữ liệu, người ta thường phân chia các bài toán thành một số hướng chính dựa trên mục tiêu và phương pháp xử lý, bao gồm:

- Tập phổ biến và luật kết hợp (Frequent Itemsets & Association Rules): Tập trung tìm kiếm các tập mục (itemset) xuất hiện thường xuyên cùng nhau trong cơ sở dữ liệu giao dịch, từ đó sinh ra các quy tắc kết hợp biểu diễn mối quan hệ giữa các mục. Ví dụ, “nếu một khách hàng mua bia thì thường mua bánh mì”.
- Tập thô (Rough Set): Sử dụng lý thuyết tập thô để giảm thiểu thuộc tính, trích xuất quy luật và xử lý dữ liệu không chắc chắn. Điều này giúp xác định tập con thuộc tính nhỏ nhất vẫn giữ nguyên khả năng phân loại dữ liệu.

- Phân lớp và dự đoán (Classification & Prediction): Mục tiêu là gán nhãn các đối tượng vào các lớp đã biết trước dựa vào các dữ liệu đầu vào (học có giám sát). Phân loại thường dùng các thuật toán cây quyết định, Naïve Bayes, mạng nơ-ron, ... để dự đoán hoặc xác định nhãn cho đối tượng mới.
- Phân cụm (Clustering): Nhóm các đối tượng thành các cụm sao cho đối tượng trong cùng cụm giống nhau hơn và khác với đối tượng ở cụm khác. Đây là bài toán học không giám sát (không có nhãn sẵn).
- Khai phá chuỗi (Sequential/Temporal Patterns): Tìm kiếm các mẫu xuất hiện theo thứ tự hoặc theo thời gian trong dữ liệu (ví dụ mua hàng theo chuỗi thời gian).
- Mô tả khái niệm (Concept Description): Tập trung mô tả và tổng hợp các đặc trưng chung hoặc quy luật trong dữ liệu mà không nhất thiết phân loại hay dự đoán.

Trong khuôn khổ đề án này, tôi tập trung vào bốn hướng cơ bản của khai phá dữ liệu: tập phổ biến và luật kết hợp, tập thô, phân lớp và phân cụm. Các hướng khác (như khai phá chuỗi hoặc mô tả khái niệm) được đề cập nhưng không triển khai chi tiết trong công việc này.

2.2.1. Tập phổ biến và luật kết hợp

Hướng **tập phổ biến và luật kết hợp** tập trung vào việc tìm kiếm các tập mục (itemset) xuất hiện thường xuyên trong cơ sở dữ liệu giao dịch và phát hiện các quy tắc liên hệ giữa chúng. Xét bộ ba (O, I, R) biểu diễn bối cảnh khai phá dữ liệu: O là tập các giao dịch, I là tập các mục (items), và $R \subseteq O \times I$ là quan hệ cho biết giao dịch nào chứa mục nào. Cho tập $S \subseteq I$, **độ phổ biến** (support) của S được định nghĩa là tỉ lệ giao dịch chứa S :

$$SP(S) = \frac{|\{o \in O \mid S \subseteq o\}|}{|O|}$$

Một tập S được gọi là **phổ biến** (frequent itemset) nếu $SP(S) \geq \text{minsup}$ (mức ngưỡng tối thiểu cho trước).

Với các bài toán kiểu phân tích giỏ hàng, dữ liệu thường được biểu diễn dưới dạng tập các giao dịch (transactions), mỗi giao dịch chứa một tập các mục (items). Một tập mục (itemset) được gọi là phổ biến nếu nó xuất hiện trong số giao dịch lớn (hỗ trợ cao) hơn ngưỡng cho trước. Cụ thể, độ hỗ trợ (support) của tập mục X được định nghĩa là tỉ lệ các giao dịch chứa X so với tổng số giao dịch.

$$\text{Support}(X) = \frac{\text{Số giao dịch chứa } X}{\text{Tổng số giao dịch}}$$

Ví dụ, nếu trong 100 giao dịch có 20 giao dịch chứa cả mục bánh mì và bơ, thì độ hỗ trợ của tập $\{\text{bánh mì}, \text{bơ}\}$ là 0.20. Muốn tập X được coi là phổ biến, ta cần $\text{Support}(X)$ vượt quá một ngưỡng tối thiểu minsup.

Từ các tập mục phổ biến thu được, người ta sinh ra các luật kết hợp dạng $X \rightarrow Y$, nghĩa là “nếu X xuất hiện thì Y cũng xuất hiện”. Trong đó X và Y là hai tập con không

giao nhau của một tập mục phổ biến. Độ mạnh của luật được đo bằng độ tin cậy (confidence), được định nghĩa bằng xác suất có Y trong giao dịch khi X đã có.

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

Luật kết hợp chỉ được coi là có ý nghĩa nếu độ tin cậy CF đạt trên ngưỡng tối thiểu (minconf) cho trước. Ví dụ, nếu tập {bánh mì, bơ} có độ hỗ trợ 0.20 và tập {bánh mì} có độ hỗ trợ 0.40, thì độ tin cậy của luật {bánh mì} $\rightarrow$ {bơ} bằng $0.20/0.40 = 0.50$; tức khi mua bánh mì thì có 50% khả năng mua bơ. Chỉ những luật có độ tin cậy vượt ngưỡng minconf cho trước mới được giữ lại vì coi là đủ mạnh.

Để tìm các tập mục phổ biến, người ta thường dùng thuật toán Apriori. Apriori là thuật toán nổi tiếng do R. Agrawal và R. Srikant đề xuất năm 1994 để tìm các tập mục phổ biến trong bộ dữ liệu lớn. Tên gọi Apriori xuất phát từ việc thuật toán sử dụng nguyên lý tiên nghiệm (a priori principle): nếu một tập mục S là phổ biến thì mọi tập con của S cũng phải phổ biến. Ngược lại, nếu một tập T không phổ biến thì bất kỳ tập mục chứa T cũng không thể phổ biến. Điều này giúp giảm đáng kể không gian tìm kiếm.

Các bước chính của thuật toán Apriori như sau:

1. **Khởi tạo (tìm tập phổ biến kích thước 1):** Quét cơ sở dữ liệu, tính số lần xuất hiện (hỗ trợ) của từng mục đơn. Chọn ra những mục có độ phổ biến $\geq$ minsup tạo thành tập L1.
2. **Sinh tập ứng viên k phần tử:** Từ tập L(k-1) (các tập phổ biến có (k-1) mục), sinh ra tập ứng viên Ck gồm các tập k mục bằng cách kết hợp các tập con.
3. **Đánh giá độ phổ biến:** Đếm hỗ trợ của từng ứng viên trong Ck bằng cách quét cơ sở dữ liệu, rồi loại bỏ những ứng viên có hỗ trợ $<$ minsup. Kết quả là tập Lk gồm các tập phổ biến k mục.
4. **Lặp lại:** Tăng k lên 1 và thực hiện bước 2-3 cho đến khi không còn tập ứng viên nào mới được tạo ra (hoặc Lk rỗng).

Khi có được tất cả các tập phổ biến, thuật toán sẽ sinh các **luật kết hợp**. Với mỗi tập S phổ biến, phân S thành hai tập con X và Y ($X \cup Y = S$, $X \cap Y = \emptyset$) để tạo luật $X \rightarrow Y$. Mỗi luật thu được có độ tin cậy $CF(X \rightarrow Y)$ được tính theo công thức đã nêu, và chỉ giữ lại luật có $CF \geq$ minconf. Qua đó, Apriori cho phép rút ra một tập quy tắc kết hợp mạnh mẽ từ dữ liệu giao dịch.

2.2.2. Tập thô (Rough Sets)

Lý thuyết tập thô được đề xuất bởi Zdzisław Pawlak vào đầu thập niên 1980 và là một phương pháp toán học để xử lý dữ liệu thiếu chính xác, không chắc chắn. Khác với lý thuyết tập cổ điển, tập thô cho phép mô tả một tập mục tiêu X thông qua hai tập xấp xỉ: hợp xấp xỉ dưới chứa những đối tượng chắc chắn thuộc X, và hợp xấp xỉ trên chứa những đối tượng có khả năng thuộc X. Khoảng cách giữa hai hợp này là vùng biên (boundary), gồm các phần tử không thể khẳng định rõ ràng là thuộc hay không thuộc X.

Trong tiếp cận này, mỗi dữ liệu được mô tả bởi một hệ thông tin (information system), trong đó U là tập hợp các đối tượng và A là tập hợp các thuộc tính. Mỗi đối tượng $x \in U$ có các giá trị thuộc tính xác định.

Xấp xỉ dưới và xấp xỉ trên: Cho một tập đối tượng $X \subseteq U$ và một tập thuộc tính $B \subseteq A$, ta định nghĩa hai tập xấp xỉ của X dựa trên quan hệ tương đương (indiscernibility) do B tạo ra.

- Tập xấp xỉ dưới $\underline{B}X = \{x \mid [x]_B \subseteq X\}$ gồm các đối tượng có lớp tương đương hoàn toàn nằm trong X ; nghĩa là những đối tượng chắc chắn thuộc X .
- Tập xấp xỉ trên $\overline{B}X = \{x \mid [x]_B \cap X \neq \emptyset\}$ gồm các đối tượng có lớp tương đương giao với X không rỗng; nghĩa là những đối tượng có thể thuộc X .
- Vùng B-biên của X , $BN_B(X) = \overline{B}X - \underline{B}X$ chứa các đối tượng không thể phân lớp rõ ràng (vừa thuộc vừa không thuộc X).
- Vùng B-ngoài của X , $U - \overline{B}X$ chứa các đối tượng chắc chắn được phân lớp không thuộc về X .
- Tập X được gọi là thô (rough) nếu vùng biên của nó không rỗng, ngược lại nếu vùng biên rỗng thì X là tập rõ (crisp).

Phụ thuộc thuộc tính: Cho hai tập thuộc tính $C, D \subseteq A$, ta nói D phụ thuộc (theo mức độ k) vào C nếu giá trị của C xác định giá trị của D . Độ phụ thuộc được đo bằng công thức:

$$\gamma(C, D) = \sum_{X \in U/IND(C)} \frac{|X \cap D|}{|X|}$$

Trong đó $U/IND(C)$ là phân hoạch của U theo quan hệ tương đương do C tạo ra, và mỗi lớp X trong phân hoạch có độ lớn $|X|$. Một khái niệm cơ bản trong tập thô là quan hệ không thể phân biệt (indiscernibility): hai đối tượng được coi là tương đương nếu chúng có cùng giá trị trên tất cả thuộc tính được quan tâm. Ví dụ, nếu hai khách hàng có cùng tuổi và thu nhập, thì trong bảng dữ liệu này chúng là không thể phân biệt.

Từ các tập xấp xỉ, ta có thể sinh ra các luật quyết định (decision rules) để mô tả tri thức. Những đối tượng thuộc xấp xỉ dưới của một lớp điều kiện tạo nên các luật chắc chắn (có xác suất 100%) đối với lớp quyết định tương ứng. Các đối tượng trong vùng biên có thể sinh ra các luật không chắc chắn (có xác suất $< 100\%$). Độ chính xác của một luật hoặc một tập luật được đánh giá bằng tỉ lệ đối tượng được phân loại đúng. Lý thuyết tập thô cho phép ta rút gọn tập thuộc tính (tập lỗi) và tìm ra các quy luật tối thiểu hiệu quả nhất để phân loại dữ liệu.

Nhờ vậy, tập thô cung cấp công cụ để phân tích và trích xuất tri thức từ dữ liệu không chính xác, phân loại dữ liệu trong bảng hệ thống thông tin và tạo ra các quy tắc dựa trên các thuộc tính của đối tượng.

2.2.3. Phân lớp (Classification)

Phân lớp là bài toán học có giám sát: từ một tập dữ liệu đã biết nhãn (lớp) của đối tượng, ta học một mô hình để gán nhãn cho các đối tượng mới. Ví dụ, khi phân lớp

khách hàng, mô hình có thể phân khách vào các nhóm “khách hàng tiềm năng” hoặc “không tiềm năng” dựa trên thông tin lịch sử. Các thuật toán phân lớp phổ biến gồm Cây quyết định và Naive Bayes, SVM, k-NN, v.v.

2.2.3.1. Cây quyết định (Decision Tree)

Cây quyết định (Decision Tree) là một mô hình cây phân cấp dùng để phân loại đối tượng dựa vào các tính chất của chúng. Mỗi nút trong cây đại diện cho một phép thử (kiểm tra) trên một thuộc tính, và mỗi nhánh tương ứng với giá trị của thuộc tính đó. Phần lá của cây tương ứng với các giá trị lớp. Mô hình này học các quy tắc dạng “nếu ... thì ...” một cách tuần tự.

Quá trình xây dựng cây quyết định thường áp dụng chiến lược tham lam (greedy): từ tập dữ liệu ban đầu, tại mỗi bước chọn thuộc tính tốt nhất để phân chia sao cho tối ưu một tiêu chuẩn về độ đồng nhất của nhóm (homogeneity). Ví dụ, thuộc tính được chọn là thuộc tính có độ lợi thông tin (information gain) hoặc độ giảm entropy nhiều nhất. Sau khi chia, tiếp tục đệ quy trên mỗi nhánh con cho đến khi đạt điều kiện dừng (ví dụ: tất cả đối tượng trong nhóm đều cùng lớp hoặc không còn thuộc tính để phân chia).

Các tiêu chí đánh giá chất lượng phân chia (độ đồng nhất) thường dùng bao gồm:

- **Entropy (mức độ hỗn loạn thông tin):** Cho tập dữ liệu S có n lớp, entropy được tính bởi:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

trong đó p_i là tỉ lệ phần trăm đối tượng thuộc lớp i trong S . Entropy càng nhỏ nghĩa là tập S càng đồng nhất (gần như cùng một lớp).

- **Độ lợi thông tin (Information Gain):** Áp dụng trong thuật toán ID3/C4.5, đo mức độ giảm entropy sau khi phân chia theo thuộc tính A . Công thức:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

với S_v là tập con của S có thuộc tính $A=v$.

- **Chỉ số Gini (Gini Index):** Được dùng trong thuật toán CART, đo độ hỗn loạn của tập S bằng:

$$Gini(S) = 1 - \sum_{i=1}^n p_i^2$$

Trong đó p_i là tỉ lệ của đối tượng thuộc lớp i trong tập hợp S . Thuộc tính phân chia được chọn là thuộc tính sau khi chia có chỉ số Gini của các nhóm con nhỏ nhất (tăng độ đồng nhất tối đa).

Bằng cách lặp lại quá trình chọn thuộc tính tối ưu và chia nhánh, ta thu được cây quyết định cuối cùng, trong đó mỗi nhánh lá đại diện cho một quy tắc phân loại. Mô

hình cây quyết định dễ hiểu và trực quan, giúp xác định các quy luật đơn giản về cách phân lớp dữ liệu.

2.2.3.2. *Naive Bayes*

Naive Bayes là phương pháp phân loại dựa trên xác suất, dùng Định lý Bayes và giả định các đặc trưng (thuộc tính) đầu vào độc lập với nhau. Theo Định lý Bayes, xác suất hậu nghiệm của một sự kiện khi biết một sự kiện liên quan xảy ra được tính bằng:

$$P(A | B) = \frac{P(B | A) \times P(A)}{P(B)}$$

Ví dụ, nếu A là sự kiện “có lửa” và B là “có khói”, thì $P(A|B)$ là xác suất có lửa khi biết có khói, liên hệ với tỷ lệ xảy ra của khói có lửa.

Phân lớp Naive Bayes đơn giản hóa bài toán bằng một giả định quan trọng gọi là giả định "ngây thơ" (naive assumption): Tất cả các đặc trưng trong là độc lập với nhau khi đã biết lớp.

Ví dụ, khi phân loại một email, thuật toán sẽ giả định rằng sự xuất hiện của từ "mua" là độc lập với sự xuất hiện của từ "khuyến mãi", miễn là chúng ta đã biết email đó thuộc lớp "Thư rác". Mặc dù giả định này hiếm khi đúng trong thực tế, nó giúp đơn giản hóa việc tính toán một cách đáng kể và thường cho kết quả tốt.

Với giả định này, thay vì phải tính một xác suất phức tạp của cả tập hợp đặc trưng cùng một lúc, chúng ta có thể tính xác suất của từng đặc trưng riêng lẻ $x_1, x_2, \dots, x_n$ rồi nhân chúng lại với nhau.

$$P(x_1, x_2, \dots, x_n | y) = \prod_{i=1}^n P(x_i | y)$$

2.2.4. *Gom cụm (Clustering)*

Phân cụm (Clustering) là kỹ thuật học không giám sát, mục tiêu là chia một tập dữ liệu thành các nhóm (cụm) sao cho các đối tượng trong cùng cụm có tính chất tương tự nhau hơn và khác biệt với đối tượng ở cụm khác. Các thuật toán phân cụm phổ biến bao gồm K-means và Mạng Kohonen.

2.2.4.1. *K-means*

K-means là một trong những thuật toán phân cụm cơ bản và được dùng rộng rãi. Ý tưởng chính của K-means là chia tập dữ liệu thành kk cụm, mỗi cụm có một tâm (centroid) sao cho tổng bình phương khoảng cách từ các điểm đến tâm cụm càng nhỏ càng tốt. Thuật toán lặp lại các bước sau:

1. **Khởi tạo:** Chọn ngẫu nhiên kk điểm làm tâm cụm ban đầu (hoặc chọn kk mẫu đầu tiên).
2. **Gán nhãn (phân cụm):** Với mỗi điểm dữ liệu, tính khoảng cách đến kk tâm cụm (thường dùng khoảng cách Euclid) và gán điểm đó vào cụm có tâm gần nhất.

Khoảng cách Euclid giữa hai điểm $p = (x, y)$ và $q = (x', y')$ được tính như:

$$d(p, q) = \sqrt{(x - x')^2 + (y - y')^2}$$

3. **Cập nhật tâm cụm:** Tính lại tâm của mỗi cụm bằng cách lấy trung bình (hoặc trung vị) tọa độ của tất cả các điểm hiện có trong cụm đó.
4. **Lặp lại:** Quay lại bước Gán nhãn và Cập nhật tâm cụm cho đến khi các tâm cụm không thay đổi nữa (hoặc thay đổi rất nhỏ) hoặc đạt ngưỡng số lần lặp nhất định.

Kết quả của K-means là một tập cụm, trong đó mỗi điểm thuộc cụm sao cho tổng sai số (variance) trong cụm được giảm thiểu. Thuật toán này yêu cầu xác định trước số cụm, và chất lượng phân cụm phụ thuộc vào cách khởi tạo cũng như việc lặp cập nhật trọng số.

2.2.4.2. Mạng Kohonen

Mạng Kohonen (Self-Organizing Map, SOM) là một loại mạng nơ-ron nhân tạo dùng cho học không giám sát. Mục đích của SOM là ánh xạ dữ liệu đầu vào phức tạp vào một bản đồ hai chiều, sao cho các nơ-ron gần nhau trên bản đồ sẽ phản ánh các điểm dữ liệu tương tự. Quy trình huấn luyện mạng Kohonen bao gồm các bước cơ bản:

1. **Khởi tạo:** Tạo một lưới hai chiều (map) gồm các nơ-ron, mỗi nơ-ron được gán một vectơ trọng số ban đầu ngẫu nhiên (có kích thước bằng số chiều của dữ liệu). Xác định bán kính ban đầu $N_c(0)$ của vùng lân cận.
2. **Chọn mẫu và tìm nơ-ron chiến thắng:** Đưa vào một mẫu dữ liệu $v(t)$, chuẩn hóa nếu cần. Tính khoảng cách Euclid từ $v(t)$ đến trọng số của mỗi nơ-ron. Chọn nơ-ron chiến thắng (best matching unit) là nơ-ron có khoảng cách nhỏ nhất.
3. **Cập nhật trọng số:** Cập nhật trọng số của nơ-ron chiến thắng và các nơ-ron trong vùng lân cận của nó theo công thức:

$$\omega_{ijk}(t+1) = \omega_{ijk}(t) + \eta(t) [x_k(t) - \omega_{ijk}(t)]$$

trong đó $\omega_{ijk}(t)$ là trọng số của nơ-ron tại vị trí (i, j) trên lưới tại thời điểm t , $\eta(t)$ là tốc độ học giảm dần theo thời gian. Nói ngắn gọn, nơ-ron chiến thắng và các nơ-ron lân cận được điều chỉnh để trở nên gần gũi hơn với mẫu đầu vào.

4. **Lặp:** Tăng bước thời gian t lên 1, giảm dần bán kính lân cận $N_c(t)$ và tốc độ học $\eta(t)$, rồi đưa mẫu kế tiếp vào. Lặp lại quá trình 2 - 3 nhiều lần cho đến khi mạng hội tụ hoặc đạt số vòng lặp tối đa.

Sau khi huấn luyện, mạng Kohonen tự tổ chức các nơ-ron thành một bản đồ hai chiều phản ánh phân cụm của dữ liệu. Các nơ-ron lân cận có trọng số tương tự nhau, giúp trực quan hóa các cụm tiềm ẩn và cấu trúc bên trong dữ liệu gốc.

2.3. Mô hình MVC (Model-View-Controller)

MVC là một mẫu kiến trúc phần mềm (architectural pattern) được sử dụng rộng rãi để tổ chức mã nguồn của một ứng dụng thành ba phần có liên quan với nhau. Việc phân tách này giúp quản lý sự phức tạp của ứng dụng, dễ dàng bảo trì, mở rộng và cho phép các nhà phát triển khác nhau có thể làm việc song song trên các thành phần khác nhau.

2.3.1. Các thành phần chính của MVC

Thứ nhất, **Model** là thành phần chịu trách nhiệm quản lý toàn bộ dữ liệu và logic nghiệp vụ của ứng dụng. Nó trực tiếp thực thi các thao tác trên cơ sở dữ liệu, áp đặt các quy tắc xử lý và ràng buộc dữ liệu. Một đặc tính quan trọng của Model là nó hoạt động độc lập hoàn toàn với giao diện người dùng, chỉ tập trung vào bản chất của dữ liệu và các quy trình liên quan, đồng thời có cơ chế thông báo cho các thành phần quan sát (observers) khi trạng thái dữ liệu thay đổi.

Thứ hai, **View** đảm nhận vai trò trình bày dữ liệu cho người dùng cuối và thu nhận các tương tác từ họ. Về bản chất, View là lớp giao diện người dùng (UI), có nhiệm vụ chính là hiển thị thông tin do Model cung cấp và chuyển các hành động của người dùng đến Controller. View không chứa logic nghiệp vụ và trong các triển khai hiện đại, nó thường lắng nghe các thay đổi từ Model để tự động cập nhật.

Cuối cùng, **Controller** hoạt động như một thực thể trung gian, điều phối sự tương tác giữa Model và View. Nó tiếp nhận các yêu cầu đầu vào sau đó ra lệnh cho Model thực hiện các tác vụ xử lý dữ liệu phù hợp và lựa chọn View tương ứng để trả về kết quả cho người dùng.

2.3.2. Luồng hoạt động

Luồng hoạt động trong kiến trúc MVC diễn ra theo một chu trình tuần tự. Ban đầu, người dùng tương tác với View, hành động này được View chuyển đến Controller được chỉ định. Controller tiếp nhận yêu cầu, phân tích và gọi đến các phương thức xử lý trên Model để thay đổi trạng thái dữ liệu. Sau khi Model cập nhật thành công, nó sẽ phát đi thông báo về sự thay đổi này. View, với vai trò là một "observer", sẽ nhận được thông báo, truy vấn dữ liệu mới nhất từ Model và tái kết xuất (re-render) giao diện để phản ánh trạng thái mới nhất cho người dùng.

2.3.3. Ưu và nhược điểm của kiến trúc

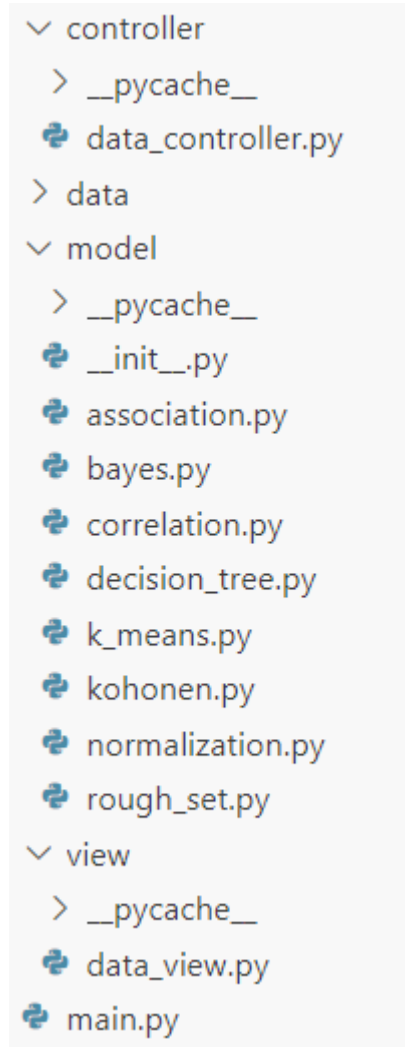
Việc áp dụng kiến trúc MVC mang lại nhiều ưu điểm đáng kể. Nó thúc đẩy một cấu trúc mã nguồn có tổ chức, minh bạch, tạo điều kiện cho việc phát triển song song giữa các nhóm front-end và back-end. Khả năng bảo trì và mở rộng cũng được cải thiện do sự tách biệt rõ ràng về chức năng. Thêm vào đó, khả năng tái sử dụng thành phần, đặc biệt là Model cho nhiều View khác nhau, giúp tối ưu hóa quá trình phát triển.

Tuy nhiên, kiến trúc này cũng tồn tại một số nhược điểm. Đối với các ứng dụng có quy mô nhỏ, đơn chức năng việc áp dụng MVC có thể làm tăng độ phức tạp không cần thiết. Nếu không phân chia các module một cách có hệ thống, luồng điều khiển và trao đổi dữ liệu giữa ba thành phần có thể trở nên rối rắm, khó theo dõi và gỡ lỗi.

Chương 3: TRIỂN KHAI CÀI ĐẶT

3.1. Cài đặt theo mô hình MVC

Các file và thư mục được tổ chức giống như trong cây thư mục trong hình sau:



Hình 1. Cấu trúc thư mục

3.1.1. Model

Chức năng: Tầng Model chịu trách nhiệm quản lý logic xử lý dữ liệu và triển khai các thuật toán học máy. Thành phần này tập trung vào các phép tính và thao tác trên dữ liệu mà không phụ thuộc vào giao diện hoặc cơ chế điều khiển, đảm bảo tính độc lập và tái sử dụng.

Các tệp liên quan:

- **correlation.py:** Thực hiện tính toán hệ số tương quan Pearson giữa hai thuộc tính thông qua hàm *calculate_correlation*.

- **normalization.py**: Thực hiện chuẩn hóa Min-Max trên các cột được chỉ định thông qua hàm *min_max_normalize*.
- **association.py**: Xử lý thuật toán Apriori tìm tập phổ biến và luật kết hợp thông qua các hàm như *apriori*, *find_maximal_frequent_itemsets*, *generate_rules*, *calculate_confidence*, và *find_satisfied_rules*.
- **roughset.py**: Triển khai các phương pháp liên quan đến lý thuyết tập xấp xỉ thô, như *lower_approximation*, *upper_approximation*, *accuracy*, *dependency_coefficient*, và *find_reducts*.
- **decision_tree.py**: Xây dựng cây quyết định sử dụng các phương pháp ID3 dựa trên Gain (*gain_id3_decision_tree*) và Gini (*gini_id3_decision_tree*), cùng với hàm trích xuất quy tắc (*extract_rules*).
- **bayes.py**: Cung cấp các hàm tính toán liên quan đến Naive Bayes, bao gồm xác suất tiên nghiệm (*calc_prior*), xác suất có điều kiện (*calc_likelihood*, *calc_likelihood_laplace*), và dự đoán (*predict_bayes*, *predict_bayes_laplace*).
- **k_means.py**: Thực hiện thuật toán phân cụm K-means tùy chỉnh thông qua hàm *k_means_custom*, hỗ trợ bởi hàm *euclidean_distance*.
- **kohonen.py**: Triển khai mô hình Kohonen Self-Organizing Map (SOM) với các hàm như *get_bmu*, *neighborhood_function*, và *kohonen_som*.

Đặc điểm: Các hàm trong tầng Model nhận dữ liệu (thường là pandas.DataFrame) và trả về kết quả xử lý (số, tập hợp, hoặc cấu trúc dữ liệu khác). Chúng không tương tác trực tiếp với giao diện người dùng.

3.1.2. View

Vai trò: Tầng View (**data_view.py**) chịu trách nhiệm hiển thị giao diện người dùng và cập nhật kết quả cho người dùng. Nó cung cấp các thành phần giao diện (widgets) và tương tác với người dùng thông qua các sự kiện (như nhấn nút).

Chi tiết triển khai:

- Sử dụng tkinter để tạo giao diện với các tab (Correlation, Normalization, Apriori, Rough Set, Decision Tree, Naive Bayes, K-means, Kohonen SOM).
- Các thành phần chính:
 - Combobox: Cho phép chọn cột (ví dụ: *corr_coll_combo*, *bayes_target_combo*).
 - Entry: Nhập giá trị thủ công (ví dụ: *value_entry* cho chuẩn hóa, *min_support_entry* cho Apriori).
 - Listbox: Cho phép chọn nhiều cột (ví dụ: *kmeans_cols_listbox*).
 - Button: Gọi các phương thức điều khiển (ví dụ: *run_correlation*, *run_kmeans*).

- Text widget: Hiển thị kết quả (*result_text*).
- Canvas: Hiển thị biểu đồ (*display_plot* với matplotlib).
- Phương thức *update_columns* cập nhật danh sách cột có sẵn dựa trên dữ liệu từ Controller.
- Phương thức *display_result* và *display_plot* hiển thị kết quả văn bản và biểu đồ.

Đặc điểm: View không xử lý logic dữ liệu mà chỉ gửi yêu cầu đến Controller và hiển thị kết quả nhận được.

3.1.3. Controller

Chức năng: Tầng Controller (**data_controller.py**) đóng vai trò trung gian giữa Model và View, điều phối luồng dữ liệu và thực thi các quy tắc nghiệp vụ. Thành phần này đảm bảo rằng các yêu cầu từ người dùng được xử lý đúng cách và kết quả được truyền tải hiệu quả.

Chi tiết triển khai:

- **Khởi tạo:** Trong phương thức khởi tạo, Controller lưu trữ tham chiếu đến View và dữ liệu (*self.data*) để quản lý trạng thái ứng dụng.
- **Phương thức chính:**
 - *load_data*: Đọc tệp CSV và cập nhật danh sách cột cho View thông qua phương thức *update_columns*.
 - *calculate_correlation*: Gọi hàm *calculate_correlation* từ Model, tạo biểu đồ sử dụng matplotlib, và hiển thị kết quả.
 - *manual_min_max_normalize*: Thực hiện chuẩn hóa Min-Max thủ công và cung cấp kết quả.
 - *apriori*: Kêu gọi các hàm trong **association.py** để tính toán tập phổ biến và luật kết hợp.
 - *rough_set*: Sử dụng các hàm trong **rough_set.py** để tính toán tập xấp xỉ và hệ số phụ thuộc.
 - *decision_tree*: Xây dựng cây quyết định với *gain_id3_decision_tree* hoặc *gini_id3_decision_tree*, đồng thời vẽ cây minh họa.
 - *bayes_predict*: Thực hiện dự đoán Naive Bayes với *predict_bayes* hoặc *predict_bayes_laplace*.
 - *k_means*: Chạy thuật toán K-means với *k_means_custom* và tạo biểu đồ nếu biểu diễn được dữ liệu hai chiều.
 - *kohonen_som*: Thực hiện SOM với *kohonen_som*, tạo lưới và ánh xạ dữ liệu.

- Xử lý lỗi: Controller thực hiện kiểm tra tính hợp lệ của dữ liệu đầu vào (ví dụ: cột tồn tại, kiểu dữ liệu phù hợp) và hiển thị thông báo lỗi qua View khi cần thiết.

Đặc điểm: Controller đảm bảo sự tích hợp giữa Model và View, xử lý các quy trình nghiệp vụ và điều phối hành động dựa trên đầu vào từ người dùng.

3.2. Chức năng: Tính tương quan

3.2.1. Mô tả

Chức năng tính tương quan hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng được xây dựng bằng thư viện tkinter, bao gồm hai Combobox (*corr_coll_combo*, *corr_col2_combo*) để người dùng chọn hai cột dữ liệu. Khi người dùng nhấn nút "Tính tương quan", phương thức *run_correlation* trong lớp DataView kích hoạt quá trình.
- Controller: Lớp DataController nhận yêu cầu từ View, kiểm tra tính hợp lệ của đầu vào (ví dụ: cột tồn tại và là kiểu số), sau đó gọi hàm *calculate_correlation* từ tầng Model. Kết quả được xử lý thêm bằng cách tạo biểu đồ phân tán (scatter plot) sử dụng matplotlib, rồi trả về View để hiển thị.
- Model: Hàm *calculate_correlation* trong tệp correlation.py sử dụng phương pháp Pearson để tính hệ số tương quan giữa hai cột, dựa trên thư viện pandas.

Quá trình này đảm bảo sự tách biệt rõ ràng giữa logic dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- Đầu vào:
 - Hai cột dữ liệu số được chọn từ tập dữ liệu CSV đã tải (yêu cầu dữ liệu phải thuộc kiểu số, được kiểm tra bởi `pd.api.types.is_numeric_dtype`).
 - Dữ liệu đầu vào phải được tải trước thông qua phương thức `load_data`.
- Đầu ra:
 - Hệ số tương quan Pearson (giá trị từ -1 đến 1) được hiển thị trong `result_text`.
 - Biểu đồ phân tán (scatter plot) hiển thị mối quan hệ trực quan giữa hai cột, với tiêu đề chứa giá trị hệ số tương quan, được vẽ trên `plot_frame`.
 - Trường hợp lỗi (ví dụ: cột không tồn tại hoặc không phải kiểu số) sẽ hiển thị thông báo qua `messagebox.showerror`.

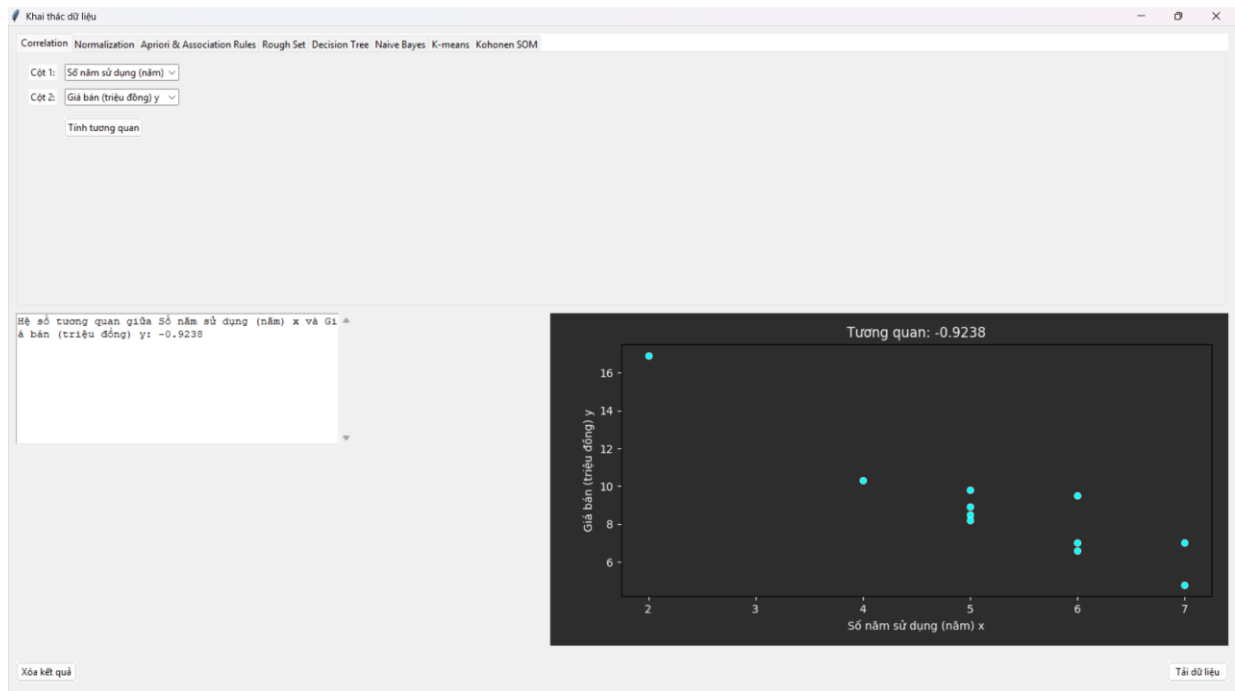
3.2.2. Giao diện và kiểm tra

Dữ liệu được sử dụng để kiểm tra là file `du_lieu_xemay_tuong_quan.csv`

Table 1. Dữ liệu trong file `du_lieu_xemay_tuong_quan.csv`

<i>STT</i>	<i>Số năm sử dụng (năm) x</i>	<i>Giá bán (triệu đồng) y</i>
<i>1</i>	5	8.5
<i>2</i>	4	10.3
<i>3</i>	6	7.0
<i>4</i>	5	8.2
<i>5</i>	5	8.9
<i>6</i>	5	9.8
<i>7</i>	6	6.6
<i>8</i>	6	9.5
<i>9</i>	2	16.9
<i>10</i>	7	7.0
<i>11</i>	7	4.8

Sau khi nhập file dữ liệu vào và chọn các cột tương ứng, kết quả được hiển thị bên dưới màn hình:



Hình 2. Kết quả thực hiện kiểm thử chức năng tính tương quan

3.3. Chức năng: Chuẩn hóa Min-Max

3.3.1. Mô tả

Chức năng chuẩn hóa Min-Max hoạt động theo nguyên tắc MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "Normalization" với các Entry để nhập giá trị cần chuẩn hóa (value_entry), giá trị nhỏ nhất và lớn nhất của thang cũ (old_min_entry, old_max_entry), cùng thang mới (new_min_entry, new_max_entry). Khi người dùng nhấn nút "Chuẩn hóa Min-Max", phương thức `run_manual_min_max_normalize` trong lớp `DataView` kích hoạt quá trình.
- Controller: Lớp `DataController` nhận dữ liệu đầu vào từ View, thực hiện kiểm tra tính hợp lệ (ví dụ: tất cả giá trị phải là số thực), sau đó gọi phương thức `manual_min_max_normalize` để tính toán giá trị chuẩn hóa. Kết quả được truyền lại cho View để hiển thị.
- Model: Hàm `min_max_normalize` trong tệp `normalization.py` xử lý trường hợp chuẩn hóa thủ công cho một giá trị duy nhất theo công thức:

$$\text{new_value} = \text{new_min} + (\text{value} - \text{old_min}) \cdot \frac{\text{new_max} - \text{new_min}}{\text{old_max} - \text{old_min}}$$

Quá trình này đảm bảo tính nhất quán và tách biệt giữa logic dữ liệu, giao diện, và điều khiển.

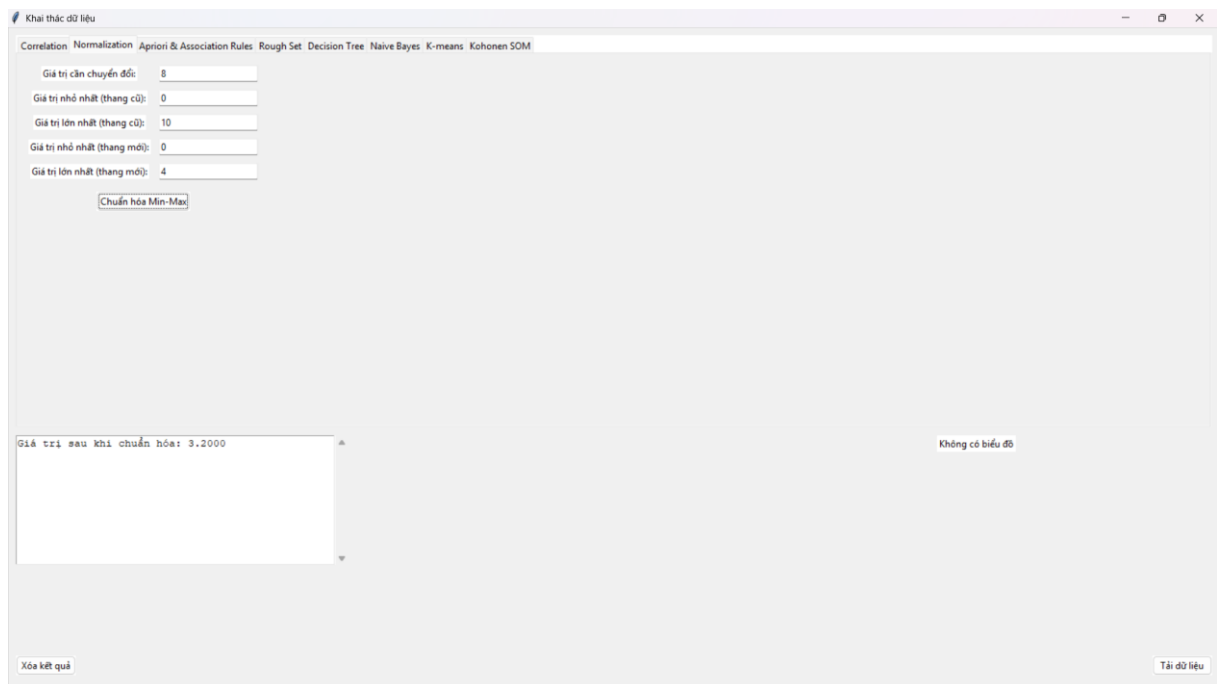
Đầu vào và đầu ra

- Đầu vào:

- Giá trị cần chuẩn hóa (value), giá trị nhỏ nhất và lớn nhất của thang cũ (old_min, old_max), cùng giá trị nhỏ nhất và lớn nhất của thang mới (new_min, new_max).
- Yêu cầu tất cả giá trị là số thực, được kiểm tra thông qua try-except trong run_manual_min_max_normalize.
- Đầu ra:
 - Giá trị đã chuẩn hóa được hiển thị trong result_text (ví dụ: "Giá trị sau khi chuẩn hóa: 0.75").
 - Trường hợp lỗi (ví dụ: old_max == old_min hoặc giá trị không phải số) sẽ hiển thị thông báo lỗi qua messagebox.showerror.

3.3.2. Giao diện và kiểm tra

Giả sử người dùng muốn chuẩn hóa giá trị 8 từ thang cũ (0 đến 10) sang thang mới (0 đến 4). Kết quả hiển thị như trên ảnh sau:



Hình 3. Kết quả thực hiện kiểm thử chức năng chuẩn hóa Min – Max

3.4. Chức năng: Tập phổ biến và luật kết hợp

3.4.1. Mô tả

Chức năng này hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "Apriori & Association Rules" với các Entry để nhập ngưỡng hỗ trợ tối thiểu (min_support_entry) và ngưỡng độ tin cậy tối thiểu (min_confidence_entry), cùng Combobox (apriori_col_combo) để chọn cột

dữ liệu (tùy chọn). Khi người dùng nhấn nút "Chạy Apriori & Association Rules", phương thức *run_apriori* trong lớp DataView kích hoạt quá trình.

- Controller: Lớp DataController nhận dữ liệu đầu vào từ View, kiểm tra tính hợp lệ (ví dụ: *min_support* và *min_confidence* phải từ 0 đến 1, dữ liệu chứa cột "Mã hóa đơn" và "Mã hàng"), sau đó gọi các hàm trong tầng Model (*apriori*, *find_maximal_frequent_itemsets*, *generate_rules*, *find_satisfied_rules*). Kết quả được truyền lại cho View để hiển thị.
- Model: Các hàm trong tệp *association.py* thực hiện thuật toán Apriori, bao gồm:
 - *apriori*: Tìm các tập phổ biến thỏa ngưỡng hỗ trợ.
 - *find_maximal_frequent_itemsets*: Xác định các tập tối đại.
 - *generate_rules*: Sinh các quy tắc kết hợp.
 - *calculate_confidence* và *find_satisfied_rules*: Tính độ tin cậy và lọc quy tắc thỏa ngưỡng.

Quá trình này đảm bảo tách biệt logic xử lý dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- Đầu vào:
 - Ngưỡng hỗ trợ tối thiểu (*min_support*) và ngưỡng độ tin cậy tối thiểu (*min_confidence*), phải là số thực từ 0 đến 1.
 - Cột dữ liệu (tùy chọn qua *apriori_col_combo*), mặc định sử dụng toàn bộ dữ liệu nếu không chọn.
- Đầu ra:
 - Danh sách các tập phổ biến thỏa *min_support* (bao gồm giá trị hỗ trợ).
 - Các tập phổ biến tối đại.
 - Tất cả các quy tắc kết hợp (bao gồm độ tin cậy).
 - Các quy tắc thỏa *min_confidence*.
 - Trường hợp lỗi (ví dụ: dữ liệu không hợp lệ) sẽ hiển thị thông báo qua *messagebox.showerror*.

3.4.2. Giao diện và kiểm tra

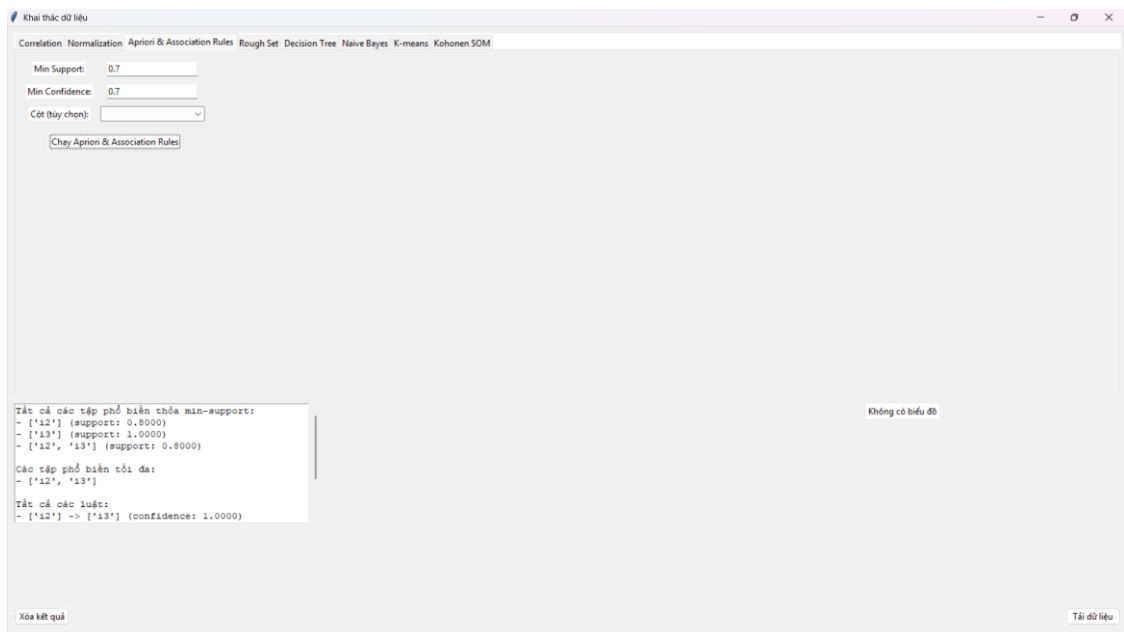
Dữ liệu được sử dụng để kiểm tra là file *du_lieu_hoa_don_tap_pho_bien.csv*

Table 2. Dữ liệu trong file *du_lieu_hoa_don_tap_pho_bien.csv*

Mã hóa đơn	Mã hàng
<i>o1</i>	<i>i1</i>

<i>o1</i>	<i>i2</i>
<i>o1</i>	<i>i3</i>
<i>o2</i>	<i>i2</i>
<i>o2</i>	<i>i3</i>
<i>o2</i>	<i>i4</i>
<i>o3</i>	<i>i2</i>
<i>o3</i>	<i>i3</i>
<i>o3</i>	<i>i4</i>
<i>o4</i>	<i>i1</i>
<i>o4</i>	<i>i2</i>
<i>o4</i>	<i>i3</i>
<i>o5</i>	<i>i3</i>
<i>o5</i>	<i>i4</i>

Tập dữ liệu chứa các giao dịch với cột "Mã hóa đơn" và "Mã hàng". Sau khi nhập file dữ liệu vào và Min Support, Min Confidence tương ứng, kết quả được hiển thị như hình sau:



Hình 4. Kết quả thực hiện kiểm thử chức năng tập phổ biến và luật kết hợp

3.5. Chức năng: Tập thô

3.5.1. Mô tả

Chức năng tập thô hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "Rough Set" với các thành phần sau:
 - num_attrs_entry: Nhập số lượng thuộc tính.
 - rough_set_attr_combos: Danh sách Combobox động để chọn thuộc tính, được cập nhật qua update_attr_combos.
 - rough_set_decision_combo: Chọn thuộc tính quyết định.
 - target_class_entry: Nhập lớp mục tiêu. Khi người dùng nhấn nút "Chạy Rough Set", phương thức *run_rough_set* trong lớp DataView kích hoạt quá trình.
- Controller: Lớp DataController nhận dữ liệu đầu vào từ View, kiểm tra tính hợp lệ (ví dụ: thuộc tính tồn tại, lớp mục tiêu hợp lệ), sau đó gọi các hàm trong tầng Model (*lower_approximation*, *upper_approximation*, *accuracy*, *dependency_coefficient*, *find_reducts*). Kết quả được truyền lại cho View để hiển thị.
- Model: Các hàm trong tệp rough_set.py thực hiện các phép tính lý thuyết tập thô, bao gồm:
 - *lower_approximation*: Tính tập xấp xỉ dưới.
 - *upper_approximation*: Tính tập xấp xỉ trên.
 - *accuracy*: Tính độ chính xác của xấp xỉ.
 - *dependency_coefficient*: Tính hệ số phụ thuộc.
 - *find_reducts*: Tìm các tập rút gọn tối thiểu.

Quá trình này đảm bảo tách biệt logic xử lý dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- Đầu vào:
 - Số lượng thuộc tính (num_attrs_entry) và danh sách thuộc tính chọn qua rough_set_attr_combos.
 - Thuộc tính quyết định (rough_set_decision_combo) và lớp mục tiêu (target_class_entry).
 - Yêu cầu tập dữ liệu đã tải, với các cột tương ứng phải tồn tại.
- Đầu ra:
 - Tập xấp xỉ dưới, tập xấp xỉ trên, độ chính xác, hệ số phụ thuộc, và các tập rút gọn được hiển thị trong result_text.

- Trường hợp lỗi (ví dụ: không chọn thuộc tính, lớp mục tiêu không tồn tại) sẽ hiển thị thông báo qua messagebox.showerror.

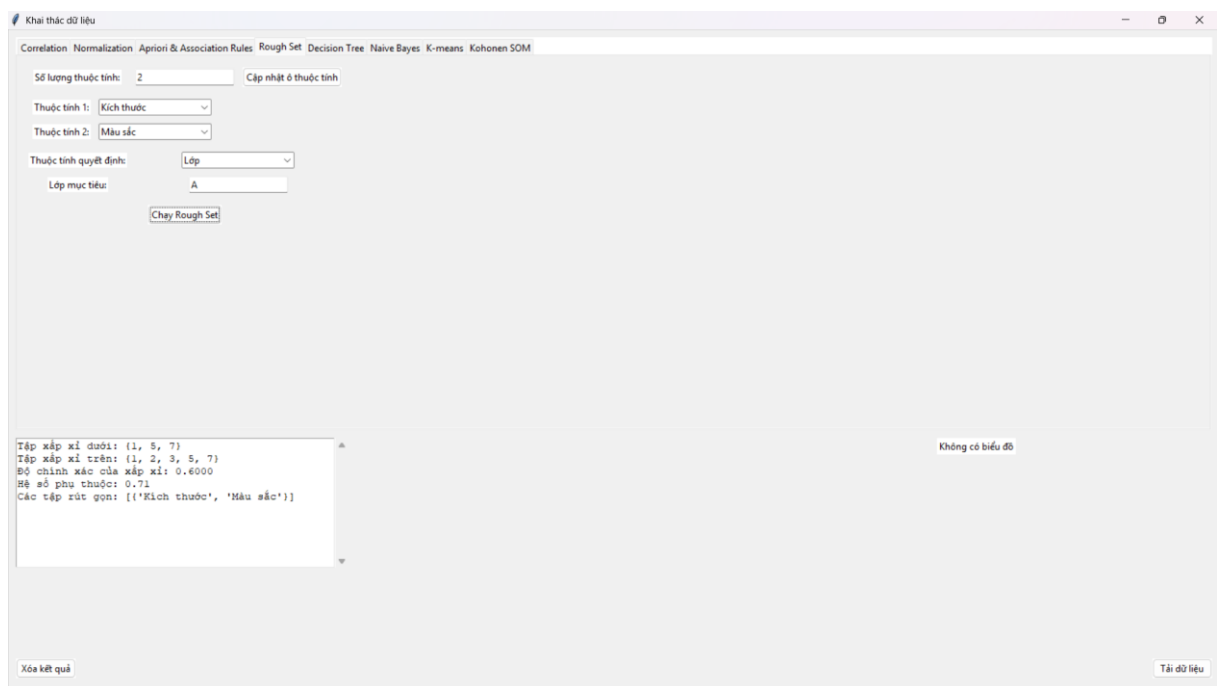
3.5.2. Giao diện và kiểm tra

Dữ liệu được sử dụng để kiểm tra là file `du_lieu_hinh_dang_tap_tho.csv`.

Table 3. Dữ liệu trong file `du_lieu_hinh_dang_tap_tho.csv`

STT	Kích thước	Màu sắc	Hình dạng	Lớp
1	Vừa	Xanh	Viên gạch	A
2	Nhỏ	Đỏ	Hình nêm	B
3	Nhỏ	Đỏ	Hình cầu	A
4	Lớn	Đỏ	Hình nêm	B
5	Lớn	Lục	Hình trụ	A
6	Lớn	Đỏ	Hình trụ	B
7	Lớn	Lục	Hình cầu	A

Giả sử tập dữ liệu chứa các cột "Kích thước", "Màu sắc", "Hình dạng", và "Lớp" với dữ liệu mẫu như trên. Sau khi nhập các thuộc tính vào ô tương ứng, chọn thuộc tính quyết định và lớp mục tiêu tương ứng và chạy, kết quả được như trên màn hình:



Hình 5. Kết quả thực hiện kiểm thử chức năng tập thô

3.6. Chức năng: Cây quyết định

3.6.1. Mô tả

Chức năng cây quyết định hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "Decision Tree" với:
 - Danh sách tree_combo (3 Combobox cho thuộc tính và 1 cho thuộc tính quyết định) để chọn các thuộc tính.
 - Hai Radiobutton (gain hoặc gini) để chọn phương pháp phân tách. Khi người dùng nhấn nút "Xây cây", phương thức *run_decision_tree* trong lớp DataView kích hoạt quá trình.
- Controller: Lớp DataController nhận dữ liệu đầu vào từ View, kiểm tra tính hợp lệ (ví dụ: thuộc tính quyết định phải tồn tại, ít nhất một thuộc tính được chọn), sau đó gọi hàm *gain_id3_decision_tree* hoặc *gini_id3_decision_tree* từ tầng Model. Kết quả bao gồm cây quyết định, các giá trị thông tin (entropy hoặc Gini), và quy tắc trích xuất được xử lý để vẽ biểu đồ bằng matplotlib, rồi truyền lại cho View để hiển thị.
- Model: Các hàm trong tệp decision_tree.py thực hiện xây dựng cây quyết định, bao gồm:
 - *calculate_entropy* và *calculate_gain*: Tính entropy và gain để chọn thuộc tính tối ưu (phương pháp Gain).
 - *calculate_gini*: Tính hệ số Gini để chọn thuộc tính tối ưu (phương pháp Gini).
 - *gain_id3_decision_tree* và *gini_id3_decision_tree*: Xây dựng cây quyết định theo phương pháp tương ứng.
 - *extract_rules*: Trích xuất các quy tắc từ cây.

Quá trình này đảm bảo tách biệt logic xử lý dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- Đầu vào:
 - Danh sách thuộc tính chọn qua tree_combo (tối đa 3 thuộc tính, không bao gồm thuộc tính quyết định).
 - Thuộc tính quyết định (target_attr) chọn từ tree_combo.
 - Phương pháp phân tách (method: "gain" hoặc "gini").
 - Yêu cầu tập dữ liệu đã tải, với các cột tương ứng phải tồn tại.
- Đầu ra:
 - Văn bản trong result_text hiển thị cây quyết định, giá trị entropy/gain hoặc Gini cho từng thuộc tính, và các quy tắc trích xuất.

- Biểu đồ cây quyết định được vẽ trên plot_frame bằng matplotlib, với các nút thuộc tính (màu cyan), lá (màu lightgreen), và nhãn trên cạnh.
- Trường hợp lỗi (ví dụ: không chọn thuộc tính hoặc thuộc tính không hợp lệ) sẽ hiển thị thông báo qua messagebox.showerror.

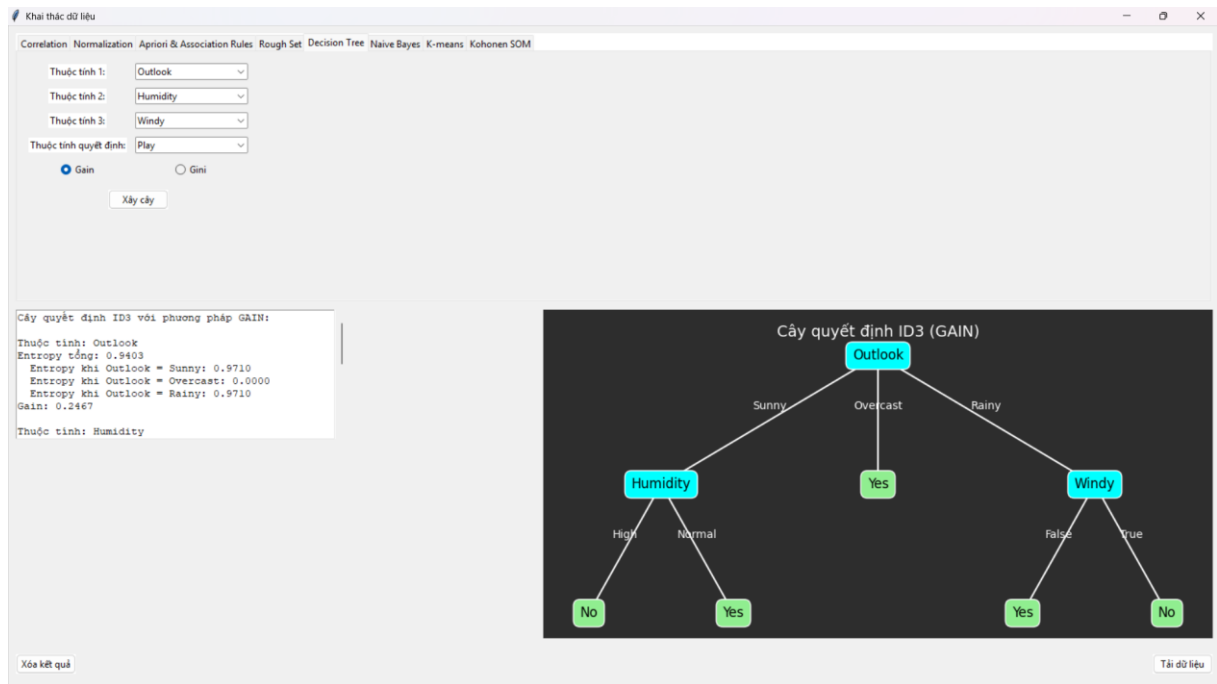
3.6.2. Giao diện và kiểm tra

Dữ liệu được sử dụng để kiểm tra là file du_lieu_thoi_tiet_cay_quyet_dinh.csv.

Table 4. Dữ liệu trong file du_lieu_thoi_tiet_cay_quyet_dinh.csv

<i>Day</i>	<i>Outlook</i>	<i>Temp</i>	<i>Humidity</i>	<i>Windy</i>	<i>Play</i>
D1	<i>Sunny</i>	<i>Hot</i>	<i>High</i>	<i>FALSE</i>	<i>No</i>
D2	<i>Sunny</i>	<i>Hot</i>	<i>High</i>	<i>TRUE</i>	<i>No</i>
D3	<i>Overcast</i>	<i>Hot</i>	<i>High</i>	<i>FALSE</i>	<i>Yes</i>
D4	<i>Rainy</i>	<i>Mild</i>	<i>High</i>	<i>FALSE</i>	<i>Yes</i>
D5	<i>Rainy</i>	<i>Cool</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D6	<i>Rainy</i>	<i>Cool</i>	<i>Normal</i>	<i>TRUE</i>	<i>No</i>
D7	<i>Overcast</i>	<i>Cool</i>	<i>Normal</i>	<i>TRUE</i>	<i>Yes</i>
D8	<i>Sunny</i>	<i>Mild</i>	<i>High</i>	<i>FALSE</i>	<i>No</i>
D9	<i>Sunny</i>	<i>Cool</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D10	<i>Rainy</i>	<i>Mild</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D11	<i>Sunny</i>	<i>Mild</i>	<i>Normal</i>	<i>TRUE</i>	<i>Yes</i>
D12	<i>Overcast</i>	<i>Mild</i>	<i>High</i>	<i>TRUE</i>	<i>Yes</i>
D13	<i>Overcast</i>	<i>Hot</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D14	<i>Rainy</i>	<i>Mild</i>	<i>High</i>	<i>TRUE</i>	<i>No</i>

Giả sử tập dữ liệu chứa các cột "Outlook", "Temp", "Humidity", "Windy" và "Play" với dữ liệu mẫu như trên. Sau khi nhập các thuộc tính vào ô tương ứng, chọn thuộc tính quyết định, phương pháp xây dựng cây và chạy, kết quả được như trên màn hình:



Hình 6. Kết quả thực hiện kiểm thử chức năng cây quyết định

3.7. Chức năng: Phân lớp Naive Bayes

3.7.1. Mô tả

Chức năng Naive Bayes hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "Naive Bayes" với:
 - bayes_target_combo: Chọn cột mục tiêu (thuộc tính quyết định).
 - sample_entry: Nhập mẫu dữ liệu dưới dạng dictionary (ví dụ: {'col1': 'value1', 'col2': 'value2'}).
 - Checkbutton (laplace): Chọn chế độ làm trơn Laplace. Khi người dùng nhấn nút "Dự đoán", phương thức run_bayes trong lớp DataView kích hoạt quá trình.
- Controller: Lớp DataController nhận dữ liệu đầu vào từ View, kiểm tra tính hợp lệ (ví dụ: cột mục tiêu tồn tại, mẫu là dictionary hợp lệ), sau đó gọi hàm *predict_bayes* hoặc *predict_bayes_laplace* từ tầng Model. Kết quả dự đoán và xác suất hậu nghiệm được truyền lại cho View để hiển thị.
- Model: Các hàm trong tệp bayes.py thực hiện tính toán Naive Bayes, bao gồm:
 - *calc_prior*: Tính xác suất tiên nghiệm của từng lớp.
 - *calc_likelihood* và *calc_likelihood_laplace*: Tính xác suất có điều kiện, với hoặc không làm trơn Laplace.

- *predict_bayes* và *predict_bayes_laplace*: Dự đoán lớp dựa trên xác suất hậu nghiệm.

Quá trình này đảm bảo tách biệt logic xử lý dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- Đầu vào:
 - Cột mục tiêu (*target_col*) chọn từ *bayes_target_combo*.
 - Mẫu dữ liệu (*sample*) nhập dưới dạng dictionary, chứa các cặp thuộc tính-giá trị.
 - Chế độ làm trơn (*laplace*), mặc định là *False*.
 - Yêu cầu tập dữ liệu đã tải, với các cột tương ứng phải tồn tại.
- Đầu ra:
 - Văn bản trong *result_text* hiển thị lớp dự đoán, mẫu đầu vào, và xác suất hậu nghiệm cho từng lớp.
 - Nếu tất cả xác suất hậu nghiệm bằng 0 (không làm trơn), chọn lớp phổ biến nhất.
 - Trường hợp lỗi (ví dụ: mẫu không phải dictionary, giá trị không tồn tại) sẽ hiển thị thông báo qua *messagebox.showerror*.

3.7.2. Giao diện và kiểm tra

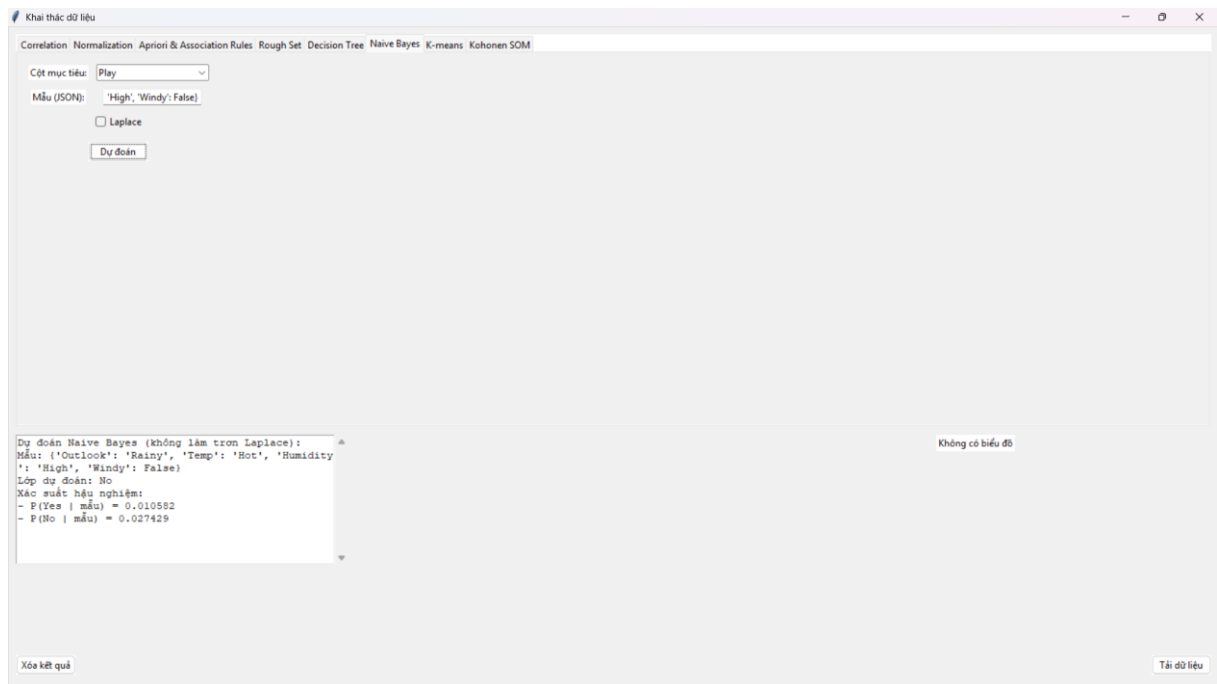
Dữ liệu được sử dụng để kiểm tra là file *du_lieu_thoi_tiet_bayes.csv*.

Table 5. Dữ liệu trong file *du_lieu_thoi_tiet_bayes.csv*

<i>Day</i>	<i>Outlook</i>	<i>Temp</i>	<i>Humidity</i>	<i>Windy</i>	<i>Play</i>
D1	<i>Sunny</i>	<i>Hot</i>	<i>High</i>	<i>FALSE</i>	<i>No</i>
D2	<i>Sunny</i>	<i>Hot</i>	<i>High</i>	<i>FALSE</i>	<i>No</i>
D3	<i>Overcast</i>	<i>Hot</i>	<i>High</i>	<i>FALSE</i>	<i>Yes</i>
D4	<i>Rainy</i>	<i>Mild</i>	<i>High</i>	<i>FALSE</i>	<i>Yes</i>
D5	<i>Rainy</i>	<i>Cool</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D6	<i>Rainy</i>	<i>Cool</i>	<i>Normal</i>	<i>TRUE</i>	<i>No</i>
D7	<i>Overcast</i>	<i>Cool</i>	<i>Normal</i>	<i>TRUE</i>	<i>Yes</i>
D8	<i>Sunny</i>	<i>Mild</i>	<i>High</i>	<i>FALSE</i>	<i>No</i>

D9	<i>Sunny</i>	<i>Cool</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D10	<i>Rainy</i>	<i>Mild</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D11	<i>Sunny</i>	<i>Mild</i>	<i>Normal</i>	<i>TRUE</i>	<i>Yes</i>
D12	<i>Overcast</i>	<i>Mild</i>	<i>High</i>	<i>TRUE</i>	<i>Yes</i>
D13	<i>Overcast</i>	<i>Hot</i>	<i>Normal</i>	<i>FALSE</i>	<i>Yes</i>
D14	<i>Rainy</i>	<i>Mild</i>	<i>High</i>	<i>TRUE</i>	<i>No</i>

Giả sử tập dữ liệu chứa các cột "Outlook", "Temp", "Humidity", "Windy" và "Play" với dữ liệu mẫu như trên. Sau khi chọn cột mục tiêu, nhập mẫu muốn phân lớp (ví dụ: {'Outlook': 'Rainy', 'Temp': 'Hot', 'Humidity': 'High', 'Windy': False}), chọn có/không làm trơn Laplace và dự đoán, ta được kết quả như hình sau:



Hình 7. Kết quả thực hiện kiểm thử chức năng phân lớp Naive Bayes

3.8. Chức năng: Phân cụm K-means

3.8.1. Mô tả

Chức năng phân cụm K-means hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "K-means" với:
 - kmeans_cols_listbox: Cho phép chọn nhiều cột dữ liệu số thông qua chế độ chọn nhiều (selectmode='multiple').

- `k_entry`: Nhập số lượng cụm (k).
- `initial_labels_entry`: Nhập nhãn ban đầu (tùy chọn, dưới dạng danh sách). Khi người dùng nhấn nút "Chạy K-means", phương thức `run_kmeans` trong lớp `DataView` kích hoạt quá trình.
- **Controller**: Lớp `DataController` nhận dữ liệu đầu vào từ View, kiểm tra tính hợp lệ (ví dụ: cột phải là số, k là số nguyên dương, nhãn ban đầu hợp lệ), sau đó gọi hàm `k_means_custom` từ tầng Model. Kết quả nhãn cụm và tâm cụm được xử lý, và nếu dữ liệu hai chiều, biểu đồ phân tán được vẽ bằng `matplotlib`, rồi truyền lại cho View để hiển thị.
- **Model**: Hàm `k_means_custom` trong tệp `k_means.py` thực hiện thuật toán K-means, bao gồm:
 - `euclidean_distance`: Tính khoảng cách Euclidean giữa các điểm.
 - Logic lặp để cập nhật tâm cụm và nhãn dựa trên khoảng cách nhỏ nhất.

Quá trình này đảm bảo tách biệt logic xử lý dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- **Đầu vào:**
 - Danh sách cột dữ liệu số chọn qua `kmeans_cols_listbox`.
 - Số lượng cụm (k) nhập qua `k_entry`, phải là số nguyên dương và không vượt quá số mẫu.
 - Nhãn ban đầu (tùy chọn) (là một list các cụm tương ứng của mẫu khởi tạo trước) nhập qua `initial_labels_entry`, phải là danh sách hợp lệ.
 - Yêu cầu tập dữ liệu đã tải, với các cột tương ứng phải là số.
- **Đầu ra:**
 - Văn bản trong `result_text` hiển thị nhãn cụm cho từng điểm dữ liệu và tọa độ tâm cụm.
 - Nếu dữ liệu hai chiều, biểu đồ phân tán (scatter plot) được vẽ trên `plot_frame`, với điểm dữ liệu theo màu cụm và tâm cụm màu đỏ.
 - Không có biểu đồ nếu dữ liệu nhiều hơn hai chiều.
 - Trường hợp lỗi (ví dụ: cột không phải số, k không hợp lệ) sẽ hiển thị thông báo qua `messagebox.showerror`.

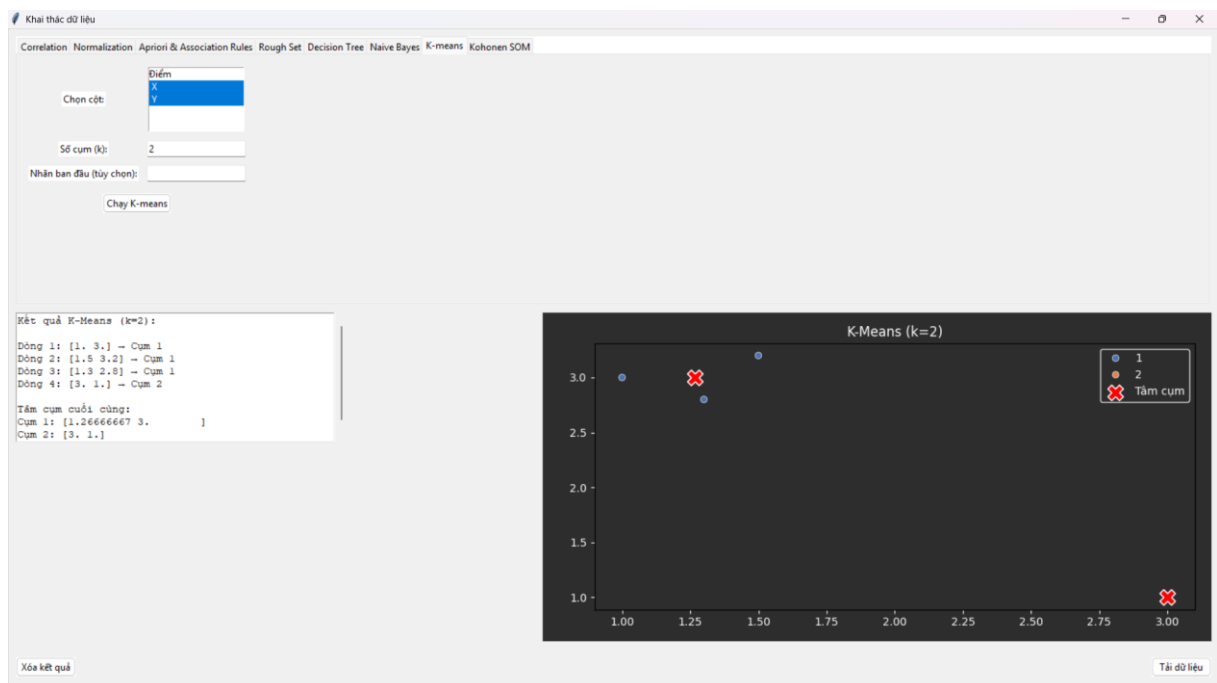
3.8.2. *Giao diện và kiểm tra*

Dữ liệu được sử dụng để kiểm tra là file `du_lieu_diem_k_mean.csv`.

Table 6. Dữ liệu trong file *du_lieu_diem_k_mean.csv*

Điểm	X	Y
X1	1.0	3.0
X2	1.5	3.2
X3	1.3	2.8
X4	3.0	1.0

Sau khi tải file dữ liệu kiểm thử vào, chọn các cột và nhập số cụm, nhấn ban đầu (tùy chọn) và chạy, ta được kết quả như hình sau:



Hình 8. Kết quả thực hiện kiểm thử chức năng phân cụm K-means

3.9. Chức năng: Phân cụm Kohonen Self-Organizing Map (SOM)

3.9.1. Mô tả

Chức năng Kohonen SOM hoạt động theo mô hình MVC như sau:

- View: Giao diện người dùng, được xây dựng bằng tkinter, cung cấp tab "Kohonen SOM" với:
 - `kohonen_cols_listbox`: Cho phép chọn nhiều cột dữ liệu số thông qua chế độ chọn nhiều (`selectmode='multiple'`).
 - `grid_shape_entry`: Nhập kích thước lưới (dạng tuple, ví dụ: (3, 3)).
 - `n_iterations_entry`: Nhập số vòng lặp.

- `initial_lr_entry`: Nhập tốc độ học ban đầu.
- `initial_sigma_entry`: Nhập sigma ban đầu. Khi người dùng nhấn nút "Chạy Kohonen", phương thức `run_kohonen` trong lớp `DataView` kích hoạt quá trình.
- **Controller**: Lớp `DataController` nhận dữ liệu đầu vào từ View, kiểm tra tính hợp lệ (ví dụ: cột phải là số, kích thước lưới là tuple hợp lệ, các tham số dương), sau đó gọi hàm `kohonen_som` từ tầng Model. Kết quả trọng số lưới và ánh xạ dữ liệu được xử lý để vẽ biểu đồ lưới bằng `matplotlib`, rồi truyền lại cho View để hiển thị.
- **Model**: Các hàm trong tệp `kohonen.py` thực hiện thuật toán SOM, bao gồm:
 - `euclidean_distance_vec`: Tính khoảng cách Euclidean giữa vector.
 - `get_bmu`: Tìm nơ-ron Best Matching Unit (BMU).
 - `decay_parameter`: Giảm tham số (learning rate, sigma) theo thời gian.
 - `neighborhood_function`: Tính hàm láng giềng Gaussian.
 - `kohonen_som`: Huấn luyện lưới SOM và cập nhật trọng số.

Quá trình này đảm bảo tách biệt logic xử lý dữ liệu, giao diện, và điều khiển.

Đầu vào và đầu ra

- **Đầu vào**:
 - Danh sách cột dữ liệu số chọn qua `kohonen_cols_listbox`.
 - Kích thước lưới (`grid_shape`) nhập qua `grid_shape_entry`, phải là tuple (height, width) với số nguyên dương.
 - Số vòng lặp (`n_iterations`), tốc độ học ban đầu (`initial_lr`), và sigma ban đầu (`initial_sigma`), phải là số dương.
 - Yêu cầu tập dữ liệu đã tải, với các cột tương ứng phải là số.
- **Đầu ra**:
 - Văn bản trong `result_text` hiển thị trọng số của từng nơ-ron trên lưới và ánh xạ dữ liệu vào các nơ-ron.
 - Biểu đồ lưới SOM được vẽ trên `plot_frame` bằng `matplotlib`, với các nơ-ron có dữ liệu ánh xạ màu skyblue, không có dữ liệu màu đen, và nhãn dữ liệu bên cạnh.
 - Trường hợp lỗi (ví dụ: cột không phải số, tham số không hợp lệ) sẽ hiển thị thông báo qua `messagebox.showerror`.

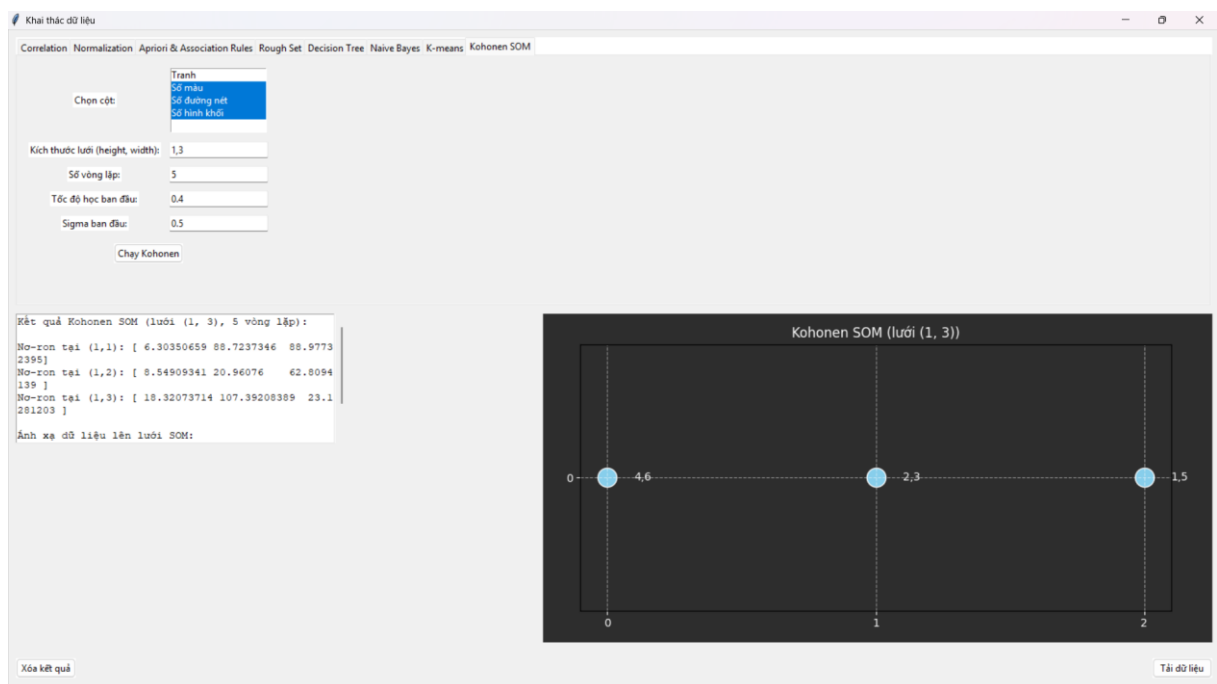
3.9.2. *Giao diện và kiểm tra*

Dữ liệu được sử dụng để kiểm tra là file `du_lieu_tranh_kohonen.csv`.

Table 7. Dữ liệu trong file *du_lieu_tranh_kohonen.csv*

<i>Tranh</i>	<i>Số màu</i>	<i>Số đường nét</i>	<i>Số hình khối</i>
1	16	124	19
2	6	13	70
3	10	22	59
4	5	81	92
5	21	97	23
6	7	94	88

Sau khi tải file dữ liệu kiểm thử vào, chọn các cột, nhập kích thước lưới, số vòng lặp, tốc độ học ban đầu, bán kính ảnh hưởng ban đầu và chạy, ta được kết quả như hình sau:



Hình 9. Kết quả thực hiện kiểm thử chức năng phân cụm Kohonen SOM

KẾT LUẬN

Đồ án “Triển khai các thuật toán khai phá dữ liệu” đã đạt được những kết quả tích cực trong việc kết hợp giữa kiến thức lý thuyết và ứng dụng thực tiễn. Qua quá trình thực hiện, tôi đã nắm vững các thuật toán quan trọng như Apriori, Rough Set, K-means, Decision Tree, Naive Bayes và Kohonen SOM, đồng thời hiểu rõ cách triển khai chúng trong một hệ thống phần mềm cụ thể. Việc áp dụng mô hình MVC và lập trình bằng Python giúp củng cố lý thuyết về khai phá dữ liệu trong thực tế. Đồ án không chỉ đóng góp về mặt học thuật khi hệ thống hóa lại các khái niệm cơ bản của khai phá dữ liệu, mà còn rèn luyện kỹ năng xây dựng phần mềm nhằm triển khai trực quan các thuật toán, qua đó nâng cao giá trị ứng dụng và khả năng thực hành môn học.

Một ưu điểm nổi bật của đồ án là giao diện thân thiện, cho phép người dùng dễ dàng tương tác và khai thác các chức năng của hệ thống, giúp đơn giản hóa quá trình khai phá, mở rộng khả năng tiếp cận cho người dung. Bên cạnh đó, thiết kế theo mô hình MVC phân tách rõ ràng các thành phần (Model, View, Controller) đã làm tăng tính module và khả năng sửa lỗi và cải thiện mã nguồn. Cấu trúc này giúp hệ thống dễ bảo trì và mở rộng: thay đổi ở một thành phần không làm ảnh hưởng đến các phần khác của ứng dụng. Hơn thế, kiến trúc tách rời còn tăng cường khả năng kiểm thử độc lập cho từng phần của hệ thống: từng module mô hình, giao diện, điều khiển có thể được kiểm tra riêng lẻ mà không ảnh hưởng lẫn nhau, từ đó nâng cao độ tin cậy của phần mềm.

Bên cạnh những ưu điểm trên, đồ án vẫn tồn tại một số hạn chế cần khắc phục trong tương lai. Hiện tại, chức năng của hệ thống còn giới hạn chỉ phục vụ các bài toán minh họa cụ thể chứ chưa đầy đủ cho tất cả các trường hợp khai phá dữ liệu. Một số tính năng (chẳng hạn như tính năng tạo cây quyết định) chưa được tối ưu hết, còn có thể cải thiện về hiệu suất và độ chính xác, cũng như là cập nhật lại giao diện tốt hơn. Đồng thời, phần mềm còn phụ thuộc nhiều vào định dạng dữ liệu đầu vào (ví dụ yêu cầu file CSV theo cấu trúc nhất định), nên chưa thật sự linh hoạt khi xử lý các tập dữ liệu phức tạp hay kém cấu trúc. Những hạn chế này phản ánh giới hạn về thời gian và phạm vi đề tài; tuy vậy, chúng cũng chỉ ra những điểm cần cải tiến để nâng cao chất lượng sản phẩm sau này.

Tóm lại, đồ án đã đạt được các mục tiêu đề ra khi kết hợp thành công giữa lý thuyết và thực hành trong lĩnh vực khai phá dữ liệu. Hệ thống xây dựng được có giao diện thân thiện, cấu trúc mô-đun rõ ràng và khả năng kiểm thử tốt, qua đó khẳng định giá trị học thuật và ứng dụng của đề tài.